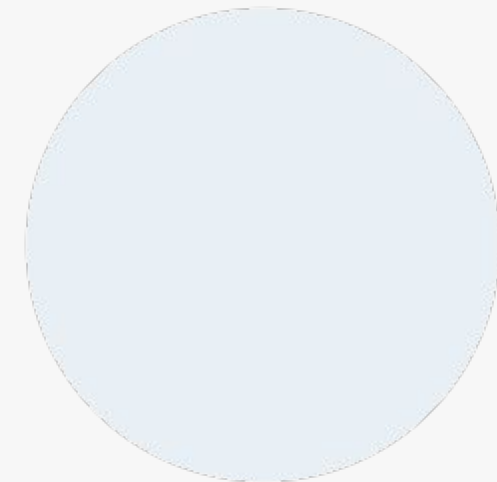


ASTON.



Автоматизация тестирования мобильных приложений



astondevs.ru

Автоматизация тестирования мобильных приложений



Сейчас невозможно представить себе жизнь без смартфона. Интернет, игры, да даже сканирование QR-кодов требует наличие этого устройства. Мобильные версии часто опережают десктопные по объему трафика. Поэтому тестирование мобильных приложений является важной частью их разработки.

И если с ручным тестированием всё понятно – жми пальцем по экрану и проверяй, то автоматизированное тестирование мобильных приложений всё ещё является чем-то новым и необычным.

Рассматривать эту тему будем на примере использования такого инструмента как **Appium**.

Что такое Appium



Appium – это кроссплатформенный инструмент для разработки автотестов для приложений на Android и iOS. Разработку тестов можно вести на различных языках программирования, но мы будем использовать Java, а тест напомним для Android-приложения.

Если вы когда-либо писали автотесты для веб-приложений с использованием Selenium WebDriver, то и Appium вам будет по плечу, так как тут используется тот же принцип: выбираем элемент по какому-то локатору и выполняем с ним требуемое действие.

Настройка окружения



Работу над написанием автотеста для мобильного приложения необходимо начать с настройки окружения для запуска тестов. Рассмотрим на примере приложения под Android.

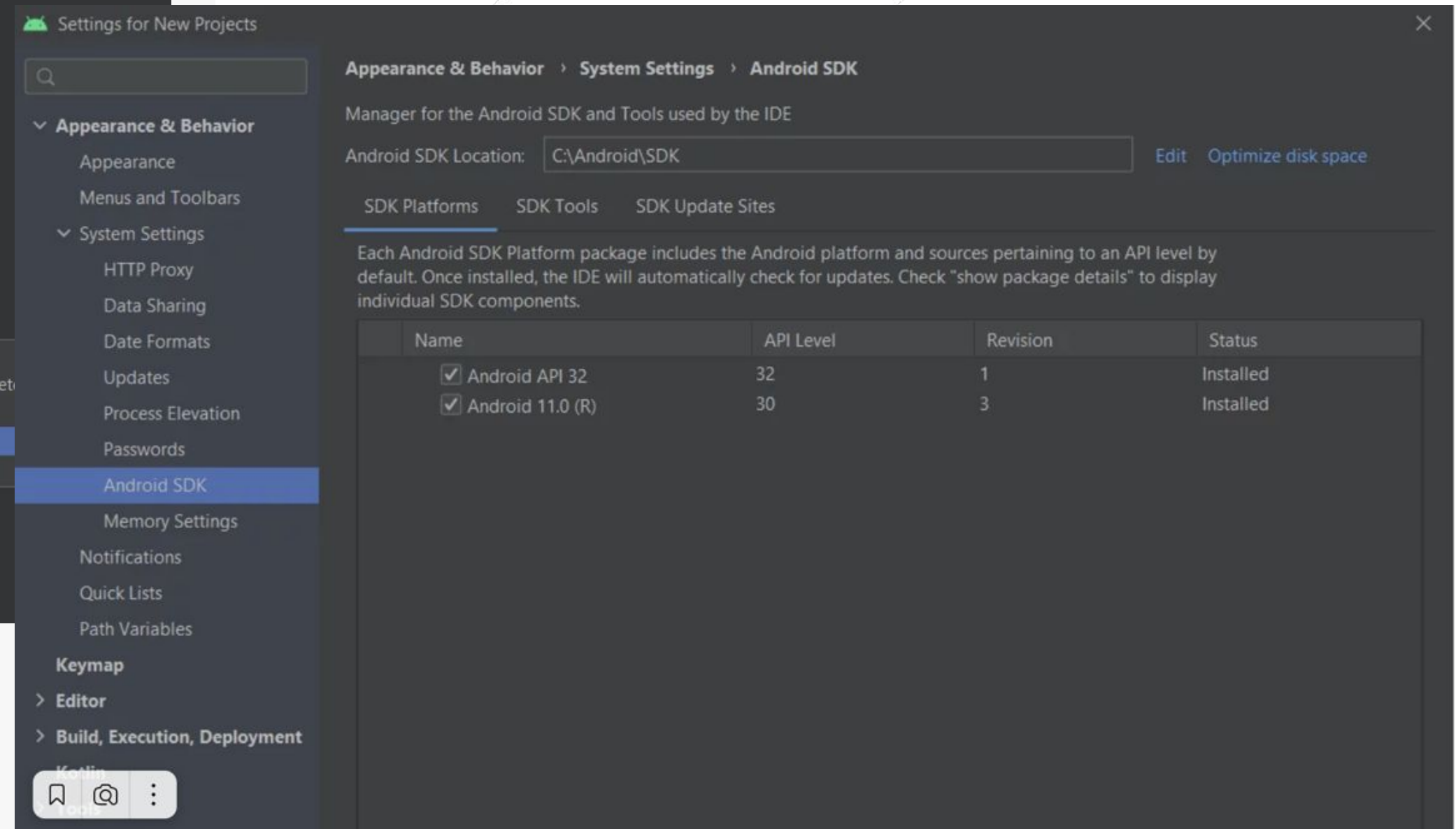
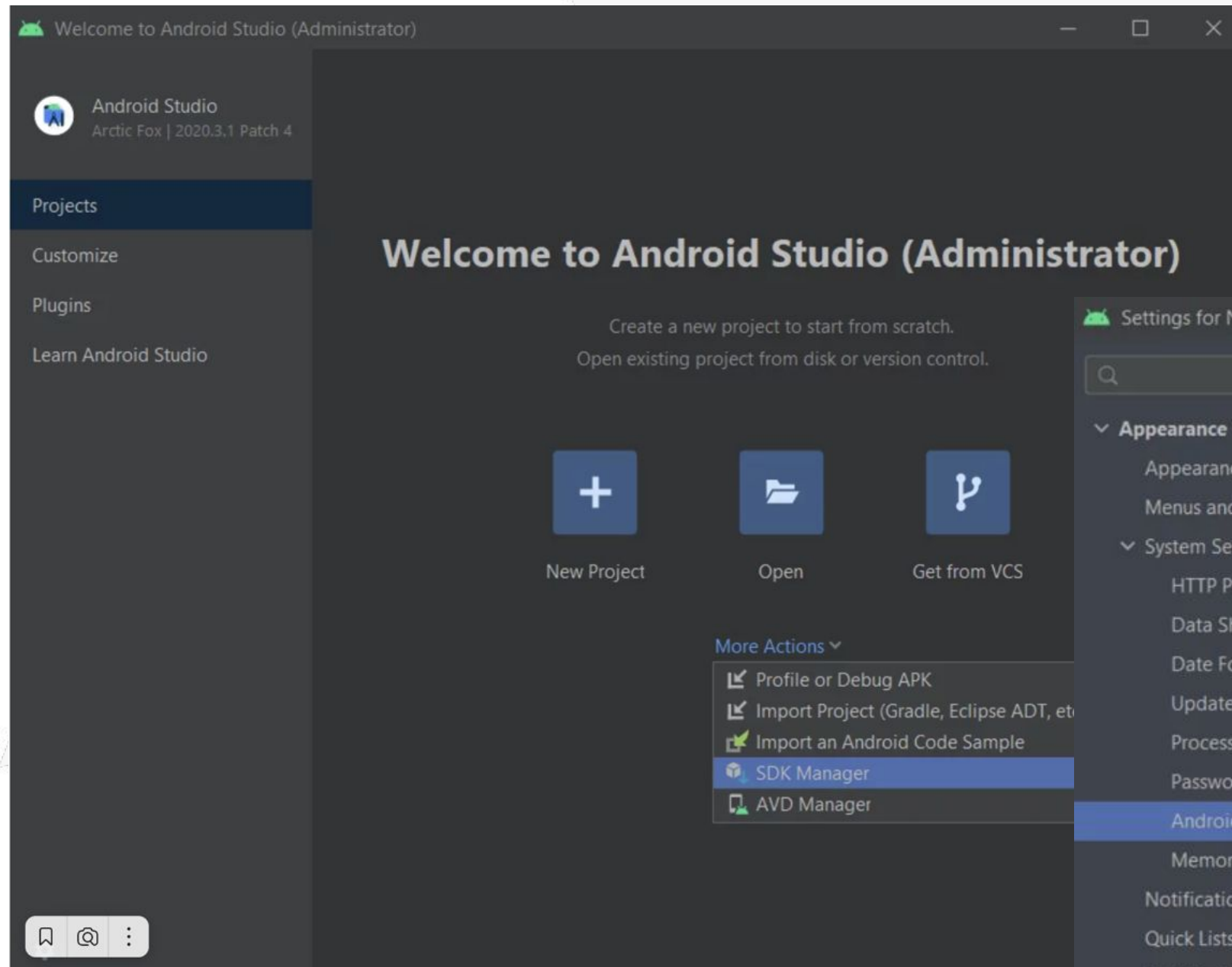
Нам понадобится:

- Java
- Android Studio
- Appium Server
- Appium Inspector
- IntelliJ Idea

Установку и настройку Java мы рассматривали ранее, потому пропустим этот шаг.

Устанавливаем Android Studio, которая включает в себя Android SDK. Обязательно проверьте, что SDK с нужной версией Android установлен, сделать это можно при запуске Android Studio через SDK Manager.

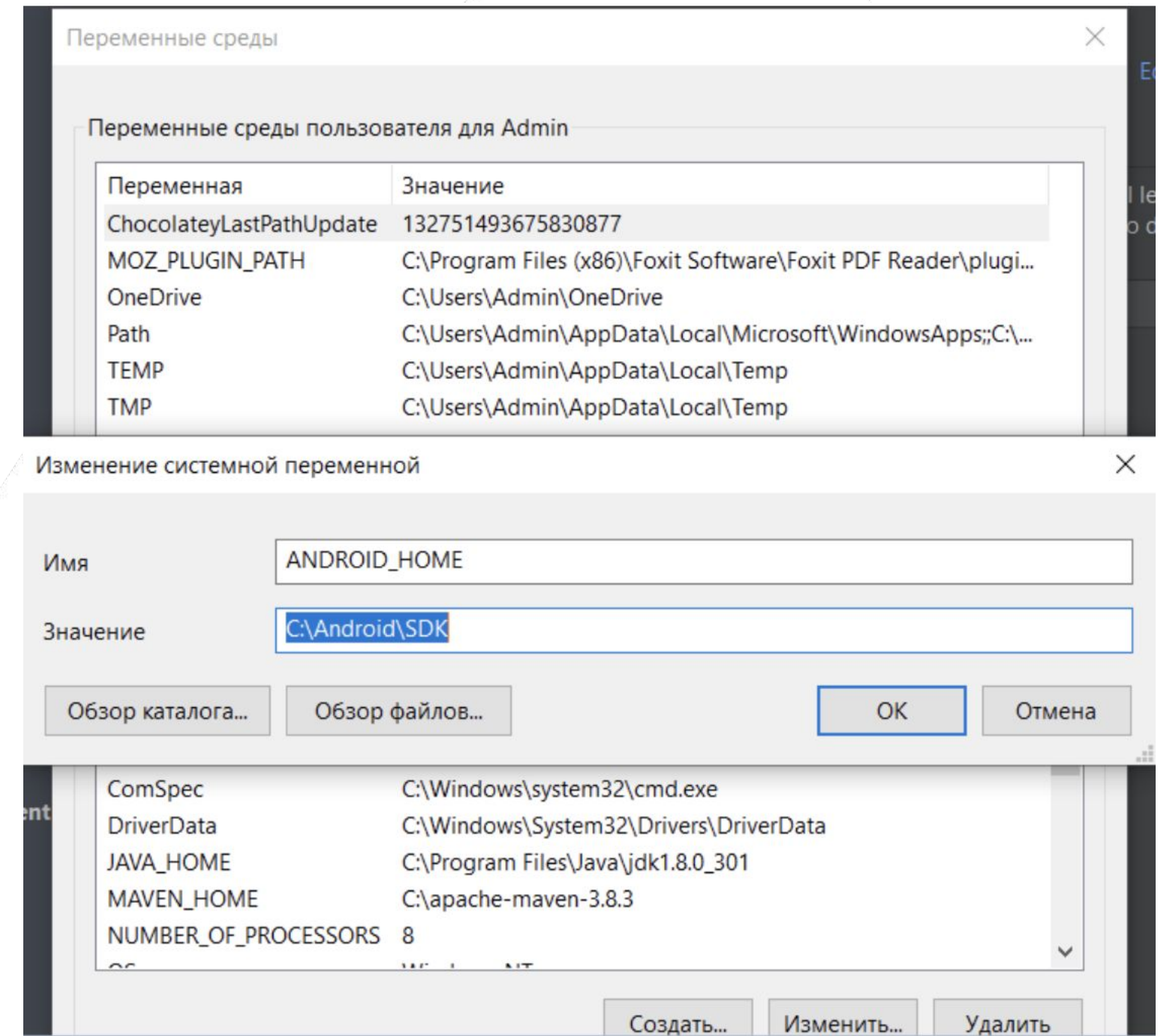
Настройка окружения



Настройка окружения

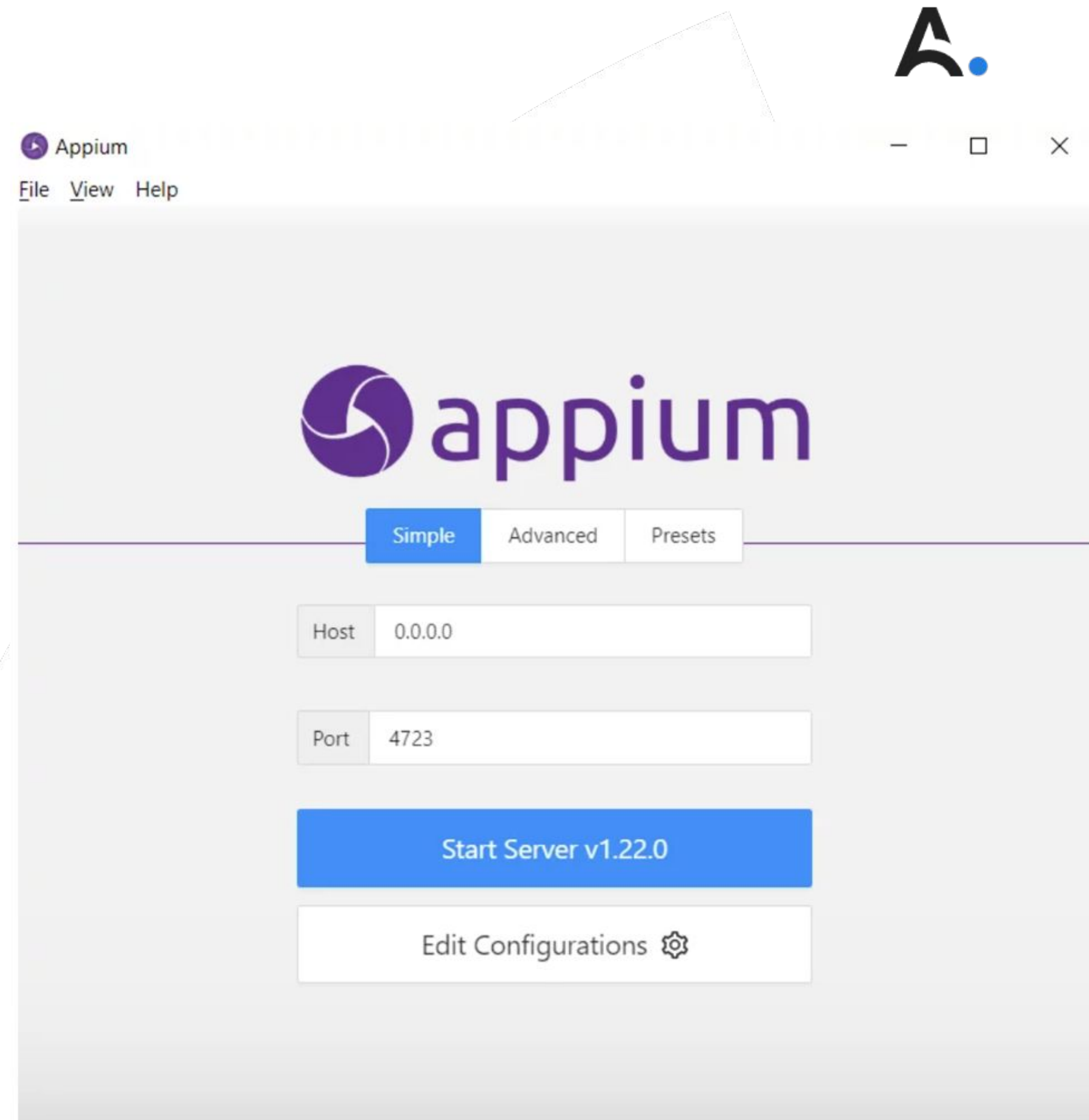


После установки добавляем системную переменную ANDROID_HOME со значением папки, в которой у нас установлен SDK, а так же обновляем переменную Path, добавляя значение %ANDROID_HOME%\bin.



Настройка окружения

После этого необходимо установить Appium Server. Это сервер, который связывает наши написанные тестовые скрипты с мобильным приложением с помощью REST API. По умолчанию сервер работает по локальному адресу 127.0.0.1 через порт 4723. Через кнопку Edit Configuration можно проверить настройки окружения, которые определяются через системные переменные. По кнопке Start Server запускаем сервер.

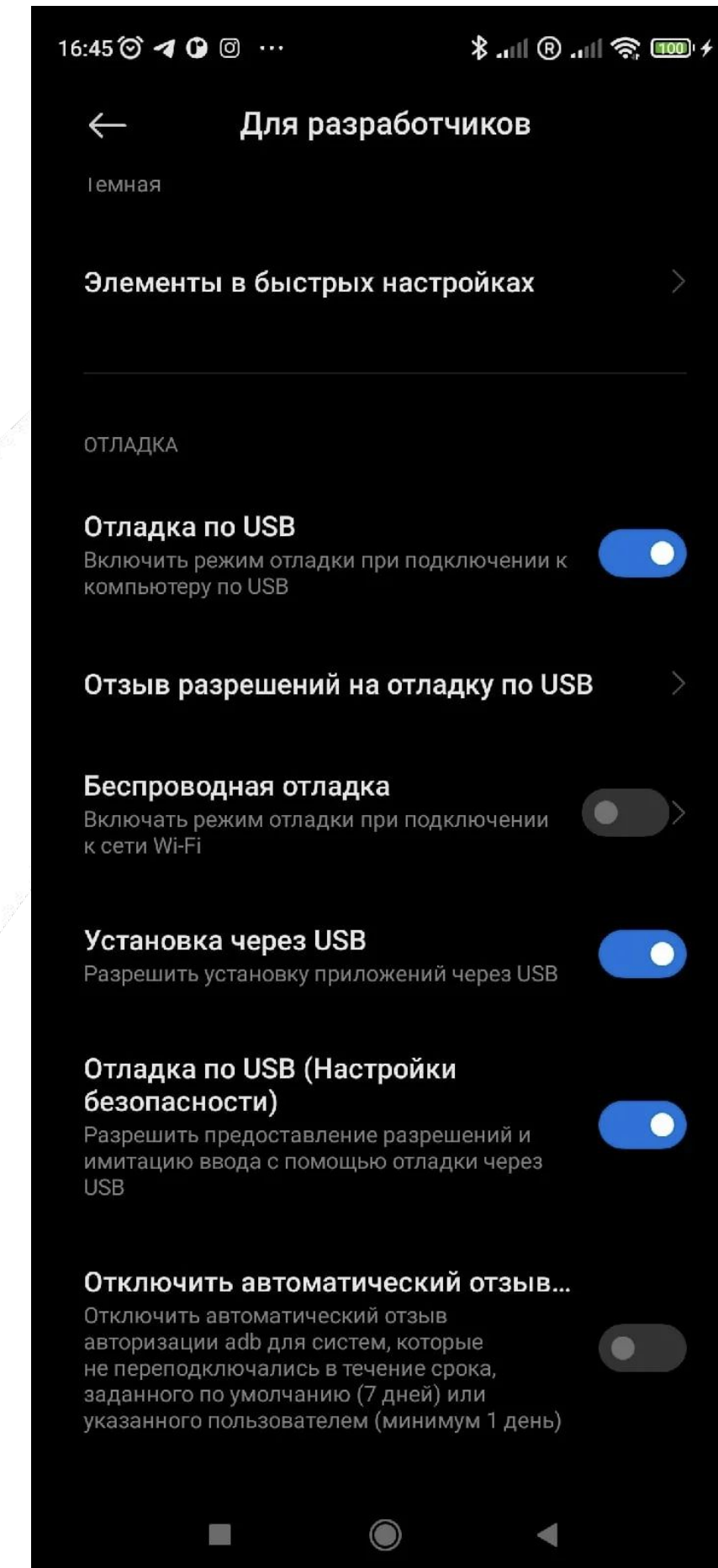


Настройка окружения

Далее нужно определиться, на каком устройстве будет выполняться тест, – реальном или виртуальном. Виртуальное устройство можно создать в Android Studio через AVD Manager, взяв параметры реального устройства или же создать своё индивидуальное.

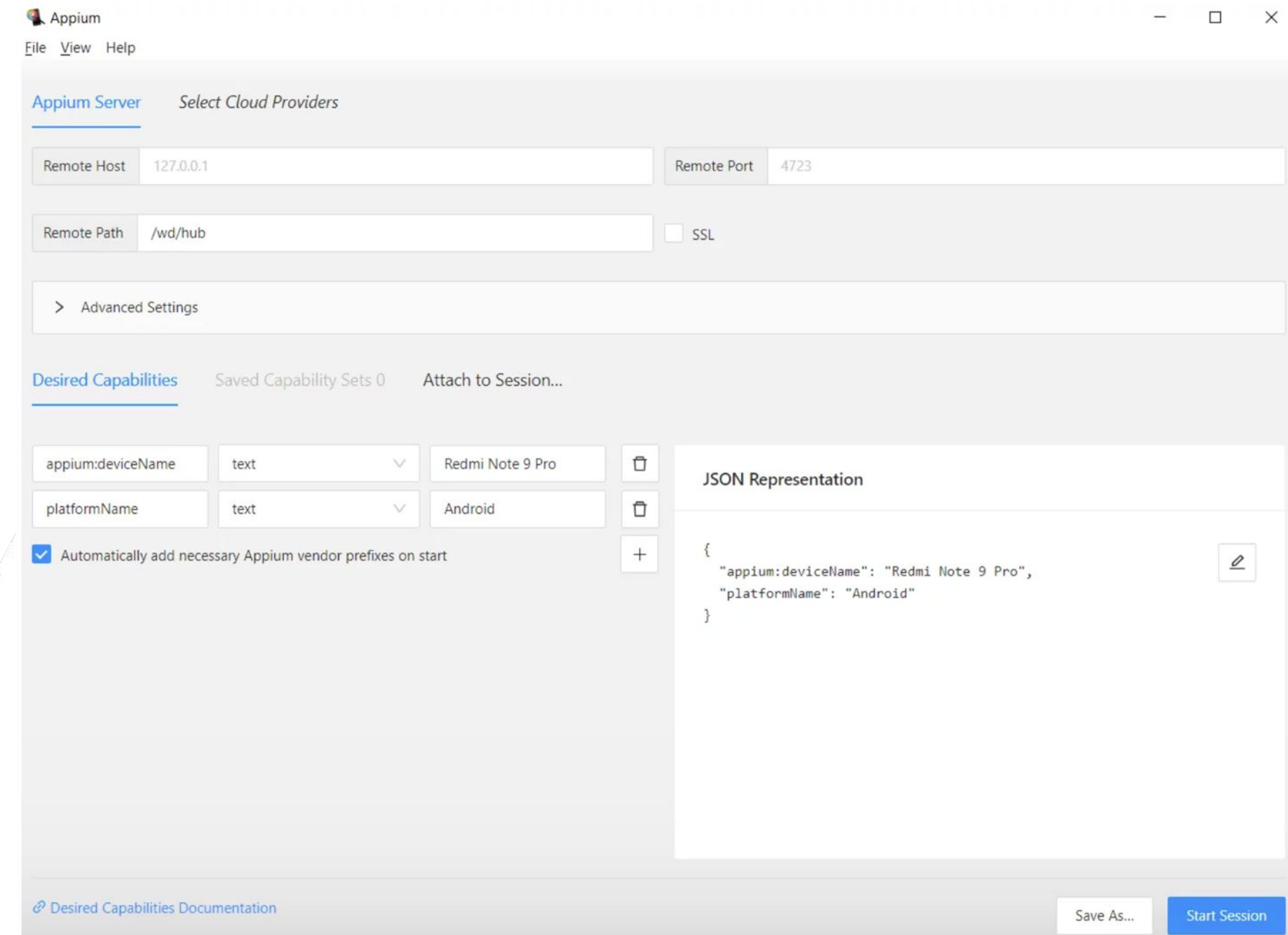
Если хотите использовать реальное устройство, то его необходимо подключить с помощью USB к компьютеру, с которого вы будете запускать тест, и нужно перейти в режим разработчика.

Для этого нужно найти в настройках телефона строку с версией Android и быстро кликать по ней, пока система не сообщит, что вы перешли в режим разработчика. После этого появится раздел настроек для разработчиков, в котором нужно активировать опции «Установка через USB» и «Отладка через USB».



Настройка окружения

Теперь нужно установить Appium Inspector. Это что-то типа панели разработчика в браузере, через которую мы определяем селекторы для html-элементов. Ранее Appium Inspector был частью Appium Server, поэтому в старых гайдах установка Appium Inspector не описана. Указываем хост и порт Appium Server, а также добавляем /wd/hub в поле Remote Path. Также нам нужно указать настройки устройства: `deviceName` – это имя вашего устройства, можно посмотреть в настройках телефона, а также `platformName` – в нашем случае это `Android`. После этого нажимаем кнопку `Start Session`, телефон подключится, и мы увидим инспектор элементов.



Настройка окружения



Appium
File View Help

Source Actions

App Source

```
<android.widget.FrameLayout>  
  <android.widget.FrameLayout resource-  
    id="com.miui.calculator:id/action_bar_overlay_layout">
```

Selected Element

Find By Selector

id	com.miui.calculator:id/btn_9_s
xpath	/hierarchy/android.widget.FrameLayout/android.widget.FrameLayout/android.widget.FrameLayout/android.widget.LinearLayout/android.widget.ViewPager/android.widget.LinearLayout/android.widget.ScrollView/android.widget.LinearLayout/android.widget.LinearLayout/android.widget.FrameLayout/android.widget.LinearLayout/android.widget.LinearLayout/android.widget.LinearLayout[3]/android.widget.LinearLayout/android.widget.LinearLayout[1]

Attribute	Value
elementId	00000000-0000-0258-ffff-ffff0000003b
index	1
package	com.miui.calculator
class	android.widget.TextView
text	9
resource-id	com.miui.calculator:id/btn_9_s
checkable	false
checked	false

Пишем первый тест



В IntelliJ Idea создаём новый проект Maven.

Затем добавляем в pom.xml зависимости для Appium и junit

```
<dependencies>
<!--
https://mvnrepository.com/artifact/io.appium/java-client
-->
  <dependency>
    <groupId>io.appium</groupId>
    <artifactId>java-client</artifactId>
    <version>7.6.0</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>RELEASE</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

В папке test/java создаем класс tests, в котором будем писать наш первый тест для мобильного приложения. В качестве «подопытного» будем использовать стандартный калькулятор. AndroidDriver вынесем отдельно как поле класса, а остальные методы для инициализации драйвера опишем в методе initialize().

Пишем первый тест



```
AndroidDriver<AndroidElement> driver = null;  
DesiredCapabilities capabilities = new  
DesiredCapabilities();
```

Теперь немного детальнее о каждом из свойств, которые нужно указать для инициализации AndroidDriver: 1) Имя устройства, на котором будет запущен тест

```
capabilities.setCapability(MobileCapabilityType.DEVICE  
_NAME, "Redmi Note 9 Pro");
```

2) Имя пакета тестируемого приложения

```
capabilities.setCapability(AndroidMobileCapabilityType  
.APP_PACKAGE, "com.miui.calculator");
```

3) Имя экрана тестируемого приложения

```
capabilities.setCapability(AndroidMobileCapabilityType  
.APP_ACTIVITY, "cal.CalculatorActivity");
```

4) Опция, которая отменяет сброс приложения до заводских настроек

```
capabilities.setCapability(MobileCapabilityType.NO_RES  
ET, true);
```

Если вы тестируете приложение, которые необходимо установить из APK-пакета, то вместо 2 и 3 строки указываете строку.

```
capabilities.setCapability(MobileCapabilityType.APP, "<  
полный_путь_к_вашему_apk>");
```

Пишем первый тест



Теперь можно запустить `AndroidDriver` с указанными свойствами и указать стандартное ожидание элементов формы. Указываем в качестве URL хост и порт нашего AppiumServer.

```
try {  
    driver = new AndroidDriver<>(new  
URL("http://127.0.0.0:4723/wd/hub"), capabilities);  
    driver.manage().timeouts().implicitlyWait(10,  
TimeUnit.SECONDS);  
} catch (MalformedURLException e) {  
    System.out.println(e.getMessage());  
}
```

Пишем первый тест



Полностью метод initialize() выглядит так:

```
public void initialize() {  
    DesiredCapabilities capabilities = new  
DesiredCapabilities();  
  
capabilities.setCapability(MobileCapabilityType.DEVICE  
_NAME, "Redmi Note 9 Pro");  
capabilities.setCapability(AndroidMobileCapabilityType  
.APP_PACKAGE, "com.miui.calculator");  
capabilities.setCapability(AndroidMobileCapabilityType  
.APP_ACTIVITY, "cal.CalculatorActivity");  
  
capabilities.setCapability(MobileCapabilityType.NO_RES  
ET, true);  
    try {  
        driver = new AndroidDriver<>(new  
URL("http://127.0.0.0:4723/wd/hub"), capabilities);  
        driver.manage().timeouts().implicitlyWait(10,  
TimeUnit.SECONDS);  
    } catch (MalformedURLException e) {  
        System.out.println(e.getMessage());  
    }  
}
```

NB: не стоит воспринимать данный пример как единственно верный.

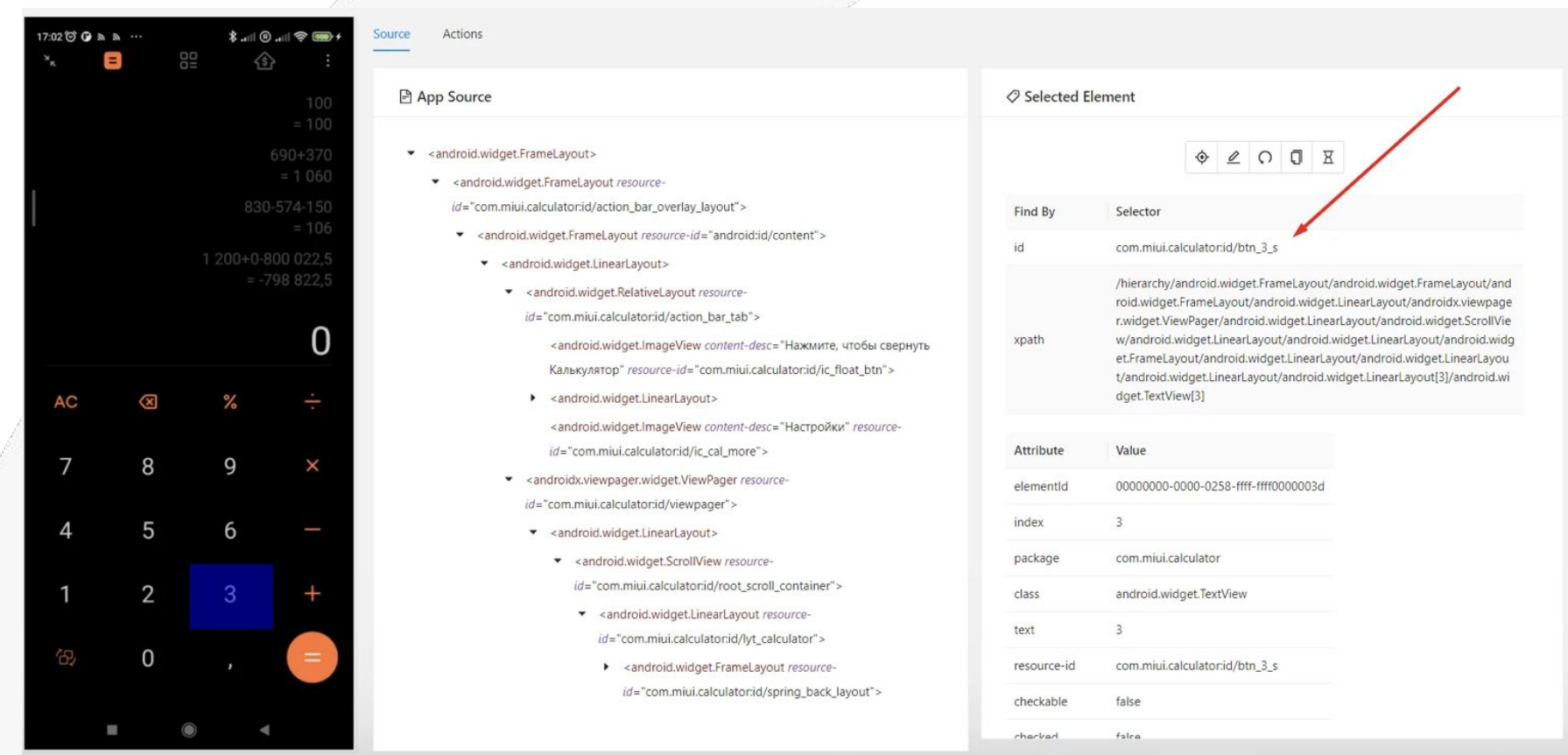
Пишем первый тест



Тест будет выполнять простейшее арифметическое действие – сложение. Будем складывать 3 и 9 чтобы получить 12. Получается, что тест должен:

1. Нажать «3»
2. Нажать «+»
3. Нажать «9»
4. Нажать «=»
5. Убедиться, что в поле вывода «12»

Чтобы определить локаторы для каждого элемента, нам надо обратиться к Appium Inspector. В идеальной картине у каждого элемента должен быть свой id. И в нашем случае он есть. Именно по этому id мы будем обращаться к кнопке «3».



Пишем первый тест



Нажатие на кнопку выполняется следующей командой:

```
driver.findElementById("com.miui.calculator:id/btn_3_s").click();
```

Очень похоже на Selenium. Тем не менее, id не всегда бывает указан в элементах, а xPath слишком громоздки, чтобы их использовать. Тогда можно определить локатор через атрибуты с помощью UIAutomator. Попробуем обратиться таким способом к кнопке через текст:

```
driver.findElementByAndroidUIAutomator("new\nUiSelector().packageName(\"com.miui.calculator\").text(\"3\").click()\n");
```

Чуть более громоздко, но это более универсальный способ определения элементов. Тем не менее, в нашем тесте нам достаточно определения по id, кроме поля с результатом. Тест будет выглядеть следующим образом:

```
@Test
public void checkCalculate() {
    initialize();
    driver.findElementById("com.miui.calculator:id/btn_3_s").click();

    driver.findElementById("com.miui.calculator:id/btn_plus_s").click();
    driver.findElementById("com.miui.calculator:id/btn_9_s").click();

    driver.findElementById("com.miui.calculator:id/btn_equal_s").click();
    assertEquals("= 12", driver.findElementByAndroidUIAutomator("new\nUiSelector().packageName(\"com.miui.calculator\").resourceId(\"com.miu\ni.calculator:id/result\")."));
}
```

Настройка xCode



Если мы имеем дело с устройствами на базе iOS, то здесь есть свои особенности. Во-первых, нам обязательно нужен MacBook (без него, увы, ничего не получится), а, во-вторых, потребуется настроить xCode.

Первый запуск

Принимаем лицензию, после чего начнется установка окружения, необходимого для работы Xcode.

Подписание WebDriverAgentRunner персональным аккаунтом разработчика

Подключаем по проводу свой iPhone. (если его пока нет, этот пункт пробуем опустить)

Открываем в Xcode проект с драйвером. Он находится здесь (Установка агента -

<https://www.npmjs.com/package/appium-webdriveragent>)

При открытии, в finder нажать command + shift + g - и ввести путь указанный ниже:

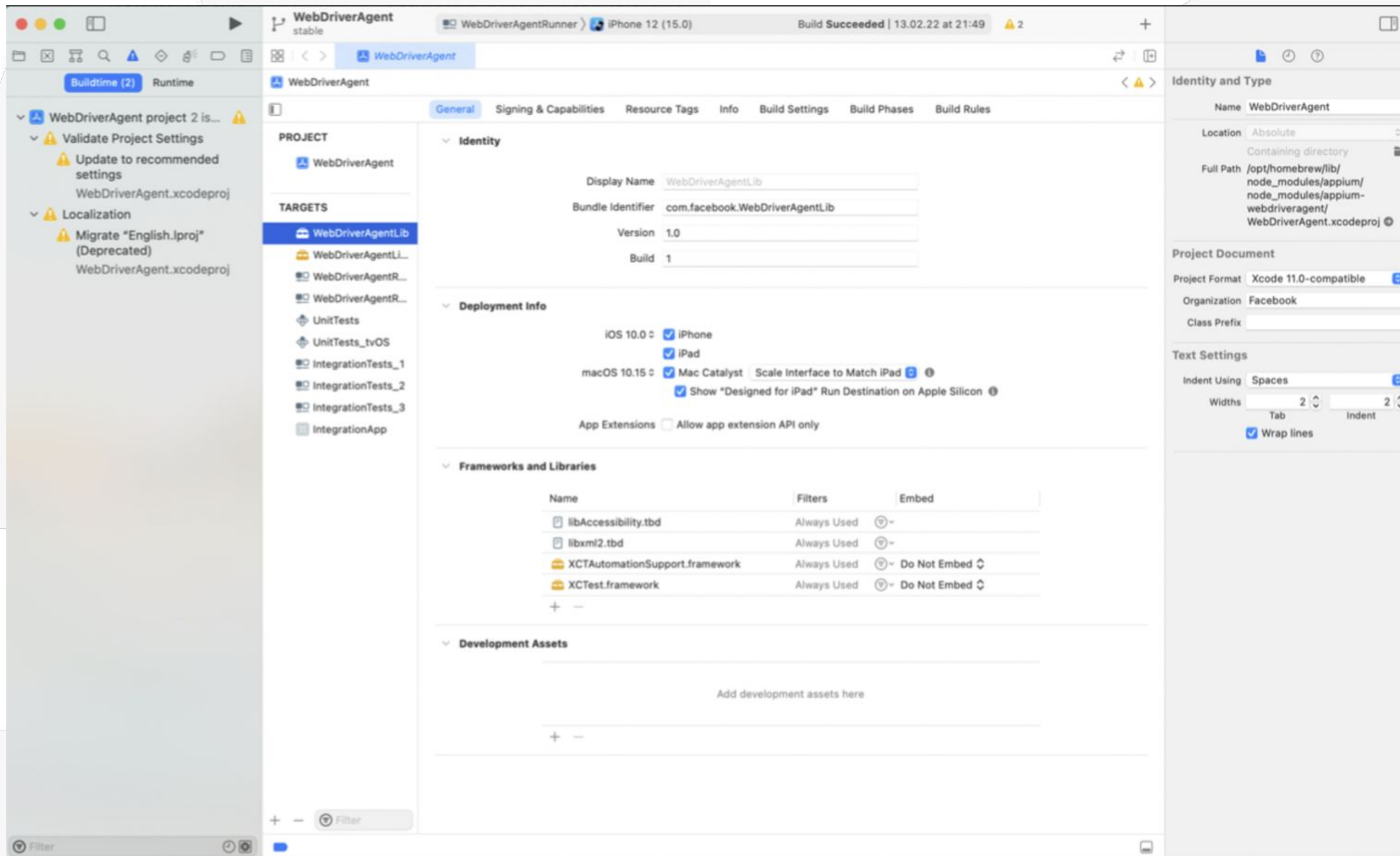
`/usr/local/lib/node_modules/appium/node_modules/appium-webdriveragent/WebDriverAgent.xcodeproj`

NB: Если для установки софта (appium) использовался

Homebrew, путь будет другой. Ищем папку

`/opt/homebrew/lib/node_modules/appium/node_modules/appium-webdriveragent/WebDriverAgent.xcodeproj`

Настройка xCode

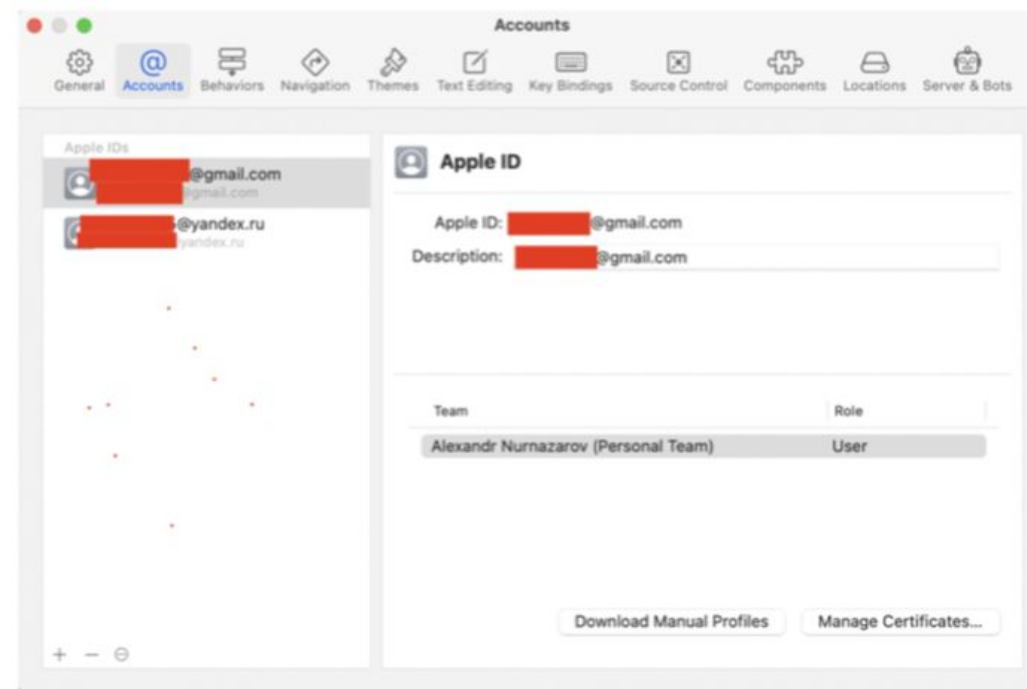
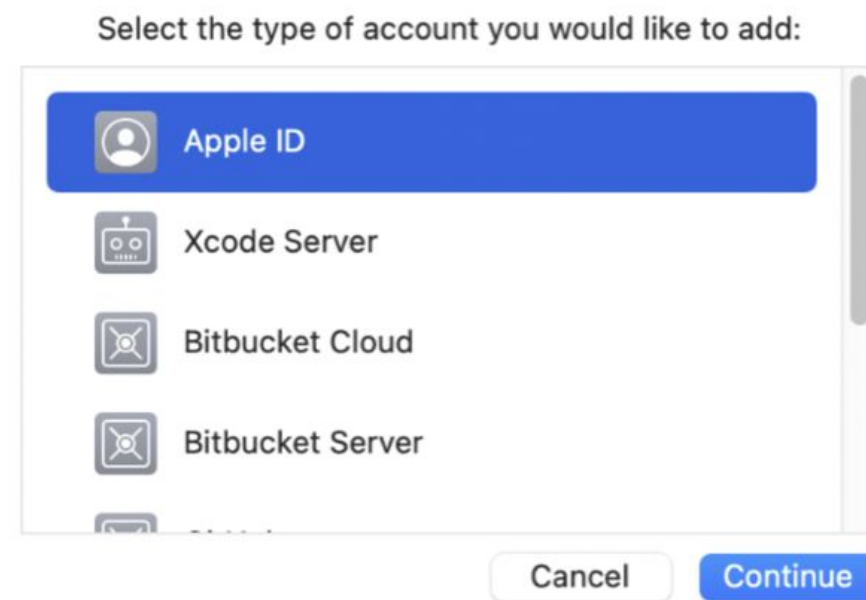


Добавляем свой Apple ID аккаунт в Xcode

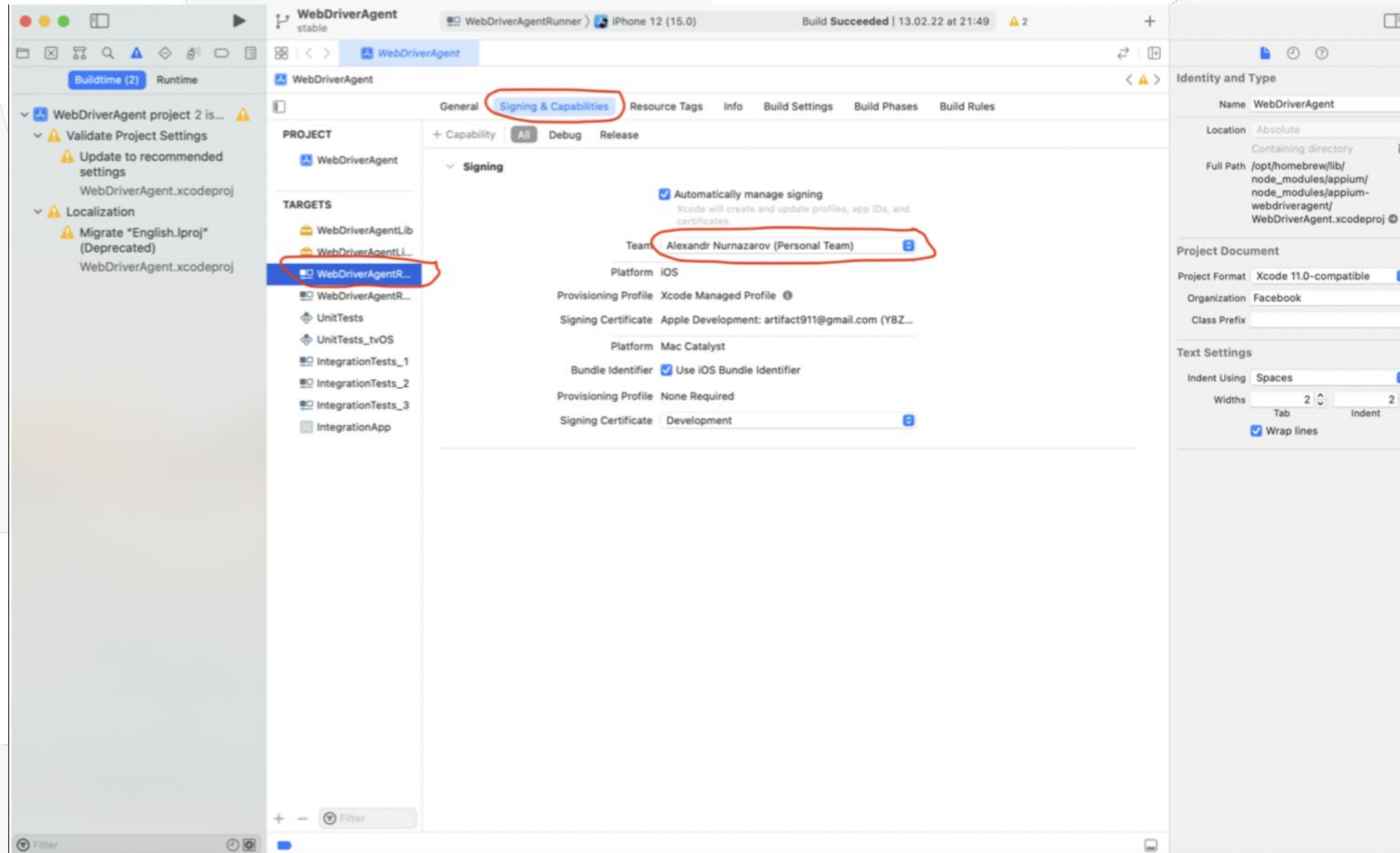


Для этого переходим в Xcode → Preferences → Accounts и нажимаем +.

Возвращаемся в проект. В разделе Targets выбираем WebDriverAgentRunner, переходим во вкладку Signing & Capabilities и в поле Team выбираем свой добавленный аккаунт.



Добавляем свой Apple ID аккаунт в Xcode



Добавляем свой Apple ID аккаунт в Xcode



Если видим ошибки связанные со значением bundle identifier, то переходим во вкладку Build Settings и меняем значение поля Product Bundle Identifier на своё уникальное.

Platform macOS

Bundle Identifier ☒ Use iOS Bundle Identifier

Provisioning Profile None Required

Signing Certificate Development

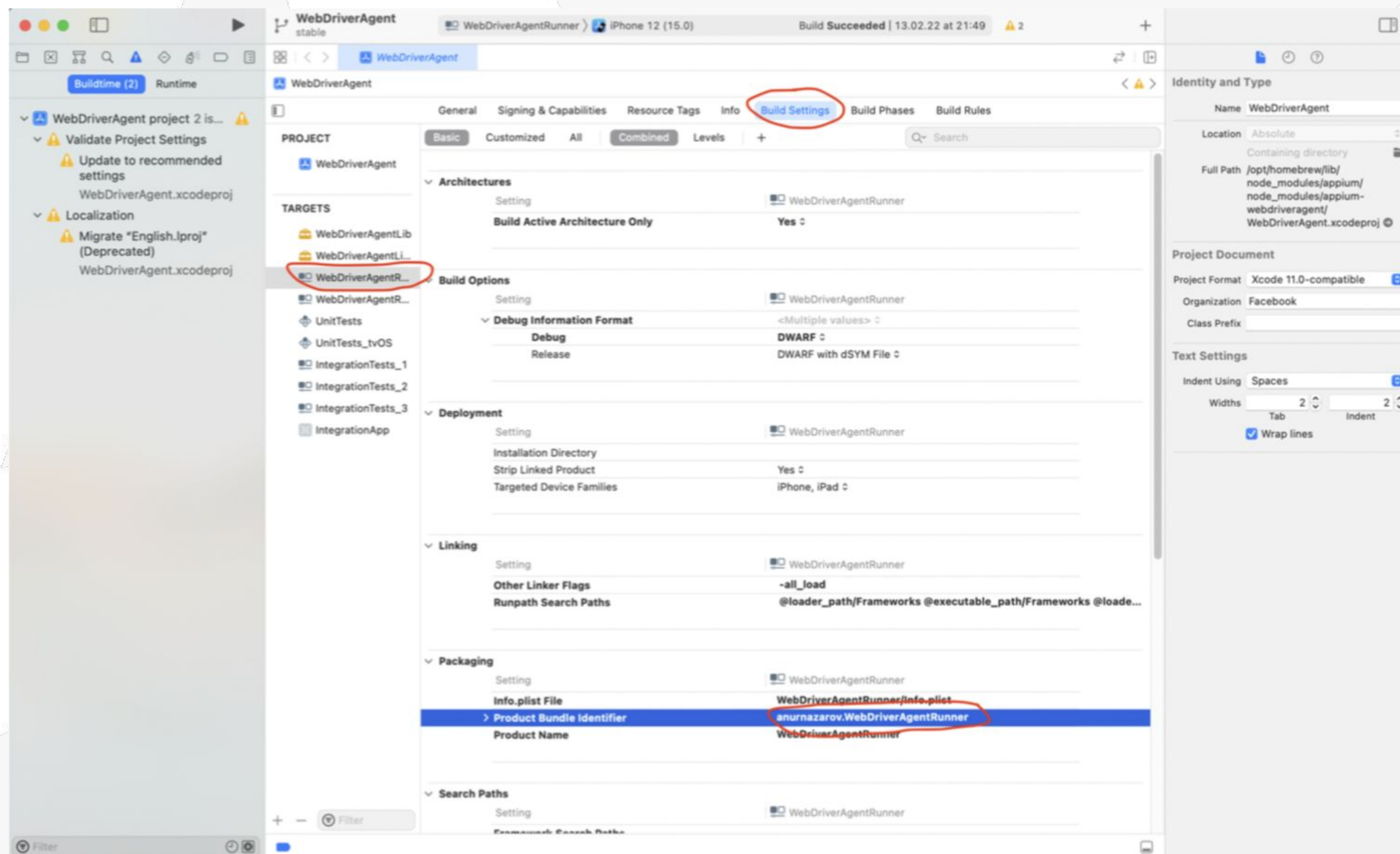
Status ❗ **Failed to register bundle identifier.**
The app identifier "com.facebook.WebDriverAgentRunner" cannot be registered to your development team because it is not available. Change your bundle identifier to a unique string to try again.
[Try Again](#)

❗ **No profiles for 'com.facebook.WebDriverAgentRunner' were found**
Xcode couldn't find any iOS App Development provisioning profiles matching 'com.facebook.WebDriverAgentRunner'.

Добавляем свой Apple ID аккаунт в Xcode



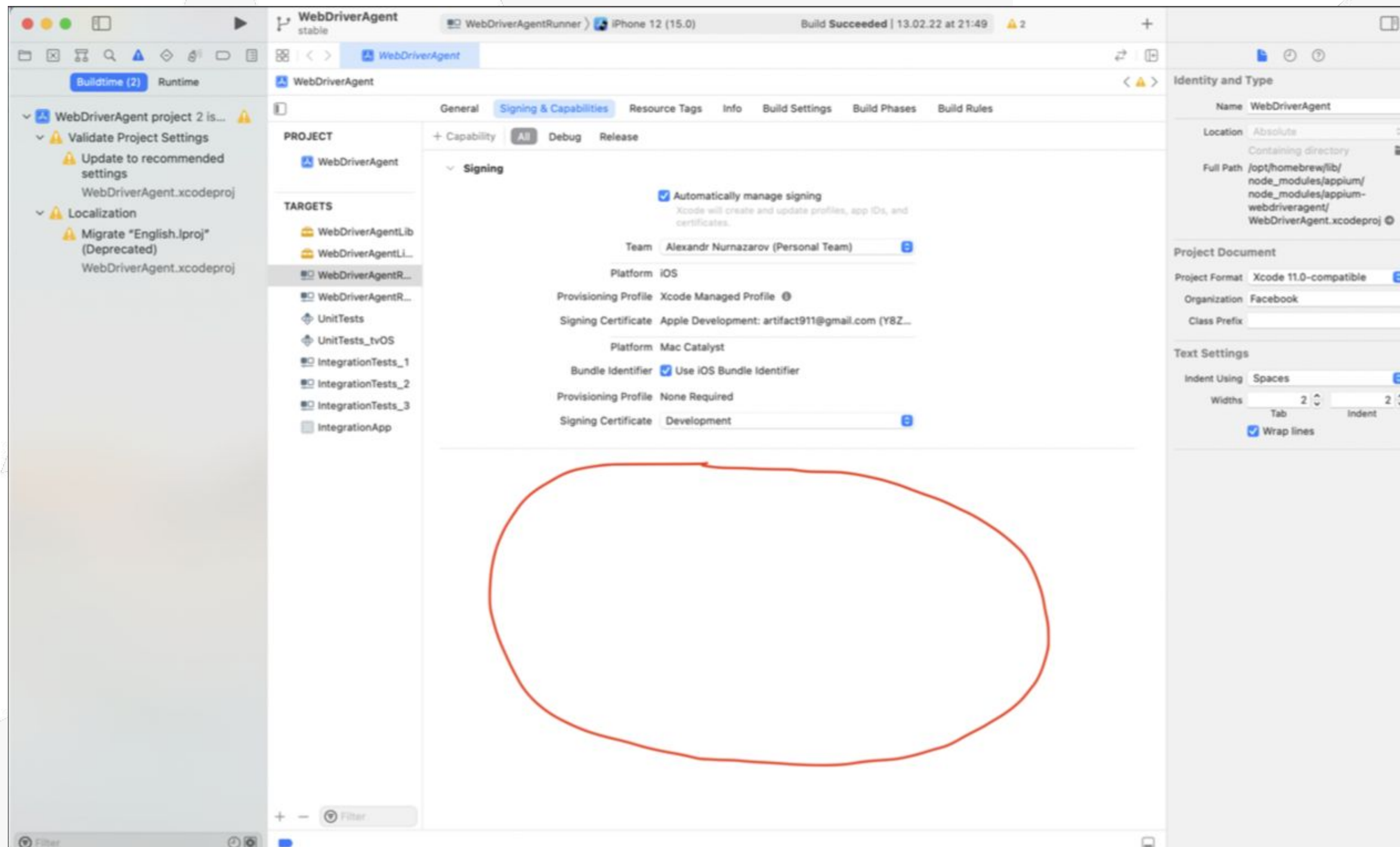
Если видим ошибки связанные со значением bundle identifier, то переходим во вкладку Build Settings и меняем значение поля Product Bundle Identifier на своё уникальное.



Добавляем свой Apple ID аккаунт в Xcode



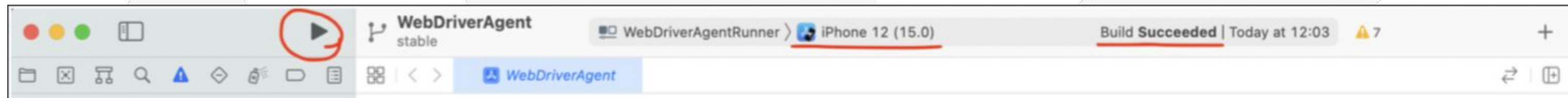
Возвращаемся во вкладку Signing & Capabilities и убеждаемся что ошибок больше нет.



Добавляем свой Apple ID аккаунт в Xcode



Теперь кликнем на значок с названием iPhone и в выпадающем меню можем выбрать модель и ось iOS. Затем кликаем на серую стрелочку слева и ждем билда. Если нашему взору явилось заветное Build Succeeded - радуемся!)



Xcode настроен!

Как открыть симулятор?



Прокликаем Xcode → Open Developer Tool → Simulator.

На симуляторе можно прокликать: File → Open Simulator → тут выбираем ОС iOS и выбираем девайс под эту ОС.

Также стоит отметить, что для iOS используется Appium XCUI Test Driver.

Практическое задание



Написать автотест для калькулятора, который совершает арифметические действия (сложение, вычитание, умножение и деление) и проверяет, что результат, отображаемый на экране соответствует ожидаемому (для теста взять приложение [«Калькулятор»](#) от Google).